

Project 1: Lossless Source Coding and Transmission of Text

曾闻天 231880147

1 Experiment Objectives

In source coding theory, Huffman coding is an optimal prefix coding method. Its core idea is to assign shorter codewords to symbols with higher frequencies and longer codewords to symbols with lower frequencies, thereby minimizing the average code length.

The objective of Project 1 is to implement a lossless text source coding system based on Huffman coding and verify its coding efficiency.

2 Experiment Process

2.1 Reading the Text Source

Function: `load_text(path: Path) -> str`

First, attempt to read the text file using UTF-8 encoding; if this fails, try using GBK encoding, treating the result as the original source symbol sequence.

2.2 Counting Symbol Frequencies

Function: `Counter(text)`

Treat the text read in 2.1 as a sequence of characters, count the frequency of each character as an estimate of the source probability distribution.

2.3 Building the Huffman Tree

Function: `build_huffman_tree(freq: Counter) -> HuffmanNode`

The construction process is divided into three stages. First is edge case handling: if the frequency table is empty, throw an exception directly; if there is only one symbol, return a leaf node directly.

Since this single-node tree is both root and leaf, special handling is required in the subsequent `build_codes` function to correctly return the code.

The second stage is heap initialization. The program traverses the frequency table, creates a `HuffmanNode` leaf node for each character, then wraps the node into a `HeapItem(freq, tie, node)` and pushes it into a min-heap. The tie starts from 0 and increments, ensuring that nodes with the same frequency have a deterministic order in the heap.

The final stage is iterative merging. As long as there is more than one element in the heap, pop the two smallest nodes, create a new internal node whose frequency is the sum of the two, with the left and right children being these two nodes. The new node is pushed back into the heap, and the tie continues to increment. Repeat this process until only one node remains in the heap –this node is the root of the entire Huffman tree.

2.4 Generating the Huffman Code Table

Function: `build_codes(root: HuffmanNode) -> Dict[str, str]`

The process of building the code table uses a classic DFS recursive traversal.

When a leaf node is reached, the current path prefix is recorded as the Huffman codeword for that character.

As mentioned earlier, there is a special case: if the entire tree has only one leaf node, meaning the input text contains only one type of character, then the prefix will remain an empty string during recursion. Since the codeword cannot be empty, it needs to be manually assigned a value of 0. For internal nodes, continue the recursive traversal: append 0 to the path when going left, and append 1 when going right.

2.5 Encoding (Compression)

Function: `HuffmanCoder.encode(text: str) -> bytes`

First, replace each character in the text with its corresponding Huffman codeword, obtaining a binary string consisting of 0s and 1s. Since each byte has 8 bits, call the `bits_to_bytes` function to pad this binary string to a multiple of 8, then pack it into a byte sequence.

At the same time, to enable correct decompression, a JSON format header needs to be generated, containing two key pieces of information: `bit_count` indicates the number of valid bits (used to remove the extra bits added during padding), and `codes` records the Huffman code table for each character.

The final output file format is: first a 4-byte magic number HUF1, followed by a 4-byte header length, then the JSON header, and finally the compressed data body.

2.6 Decoding (Decompression)

Function: `HuffmanCoder.decode(data: bytes) -> str`

When decompressing, first parse the magic number and JSON header at the beginning of the file, recovering the Huffman code table and the number of valid bits. Then call the `bytes_to_bits` function to restore the subsequent payload data to the original binary string.

Then traverse this bitstream, using the code table for reverse matching – since Huffman codes are prefix codes, each codeword can be uniquely decoded, thus restoring the original characters one by one.

To ensure that no information is lost during compression, the system also implements a lossless verification mechanism: immediately after encoding, call the decode function to decompress the compressed result and compare the decompressed result with the original text. This process is encapsulated in the `verify_lossless` function.

2.7 Efficiency Analysis

To evaluate the compression effect of Huffman coding, the system provides a series of analysis metrics.

The first is Shannon entropy, calculated via the function `shannon_entropy(freq)`,

$$H = - \sum p_i \log_2 p_i \text{ (bits/symbol),}$$

where $p_i = \text{count}_i / \text{total characters}$. It represents the theoretical optimal average code length under information theory.

Next, calculate the average code length, implemented by `average_code_length(freq, codes)`,

$$L = \sum p_i \cdot \text{len}(\text{code}_i) \text{ (bits/symbol),}$$

which is the actual average length of the encoding.

Two core metrics can be derived from this:

Efficiency $\eta = H/L$ (the closer to 1, the better the encoding) and redundancy $L - H$ (bits wasted per symbol).

In terms of compression ratio analysis, the original byte count is `len(text.encode("utf-8"))`, the compressed byte count is `len(compressed)`, and the compression ratio is defined as original bytes divided by compressed bytes. The actual bits/symbol can also be calculated, i.e., $(\text{compressed bytes} \times 8) / \text{number of characters}$, which facilitates comparison with the theoretical entropy value.

As a reference, the system uses the output of `zlib.compress(level=9)` as a benchmark for general-purpose lossless compression algorithms, thereby measuring the relative performance of Huffman coding on actual text.

2.8 Report Output

Function: `print_report(text, compressed, path)`

The output content includes:

Metric	Description
Number of symbols, unique symbols	Text length and number of character types
Shannon entropy H	Theoretical lower bound of optimal average code length
Huffman average code length L	Actual average code length
Efficiency η	H/L
Redundancy	$L - H$
Compression ratio	Original UTF-8 bytes / Huffman compressed bytes
zlib reference compression ratio	Comparison with general-purpose compressor ratio
Lossless verification	Whether the decoded result matches the original text

3 Interaction

Interactive menu (interactive_loop)

Supports three operations:

1. Encode and print report (without saving the file)
2. Encode and save a .huf file
3. Decode a .huf file and restore it to .txt

4 Experiment Results

1. Run pro1.bat, select [1] to encode and analyze test1.txt and test2.txt.

2. Program output:

```
=====
信源编码效率报告 - E:\NJU-ISE-Info Theory\pro1\test1.txt
=====
符号数 (字符)           : 5675
不同符号种类数         : 693
Shannon 熵 H (每符号)   : 8.1579 bit/符号
Shannon 熵 H (每字节)   : 5.8294 bit/字节 (UTF-8)
Huffman 平均码长 L      : 8.1914 bit/符号
冗余度 (L - H)          : 0.0335 bit/符号
编码效率  $\eta = H/L$  : 0.9959 ( $\leq 1$ , 接近 Kraft 界)
=====
原文体积 (UTF-8)        : 15969 字节
压缩后体积 (Huffman)    : 20644 字节
压缩比 (原/压)          : 0.7735
实际比特/符号 (含开销)  : 29.1017
zlib 参考               : 6180 字节 (压缩比 2.5840)
=====
无损往返验证: 通过
```

图 1: test1

```
=====
信源编码效率报告 - E:\NJU-ISE-Info Theory\pro1\test2.txt
=====
符号数 (字符)           : 5122236
不同符号种类数         : 5062
Shannon 熵 H (每符号)   : 8.7812 bit/符号
Shannon 熵 H (每字节)   : 5.7412 bit/字节 (UTF-8)
Huffman 平均码长 L      : 8.8090 bit/符号
冗余度 (L - H)          : 0.0277 bit/符号
编码效率  $\eta = H/L$  : 0.9969 ( $\leq 1$ , 接近 Kraft 界)
=====
原文体积 (UTF-8)        : 14826692 字节
压缩后体积 (Huffman)    : 5778685 字节
压缩比 (原/压)          : 2.5658
实际比特/符号 (含开销)  : 9.0253
zlib 参考               : 6059630 字节 (压缩比 2.4468)
=====
无损往返验证: 通过
```

图 2: test2

3. Result Analysis

Based on the above tests, the following conclusions can be drawn.

First, lossless verification passed: both sources were encoded and immediately decoded, and the results were completely consistent with the original text, confirming that the encoding and decoding processes are correct.

Second, the coding efficiency η is approximately 0.996 for both, indicating that the Huffman tree implementation is correct and approaches the optimal bound given by the Kraft inequality.

Third, from the perspective of size compression, for short texts (test1), the JSON code-book header overhead dominates, and a compression ratio less than 1 is expected behavior; for long texts (test2), the actual compression capability of Huffman coding on repetitive long text is fully demonstrated.

5 Conclusion

Through the above experiments, the research objectives of Project 1 have been basically achieved.